

SCHOOL OF ENGINEERING

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

COCHIN – 682022



NAME : JITHIKA P. SAJAN

BRANCH / SEM : M.E. , S-4

REG NO : 20522040

PRACTICAL RECORD

SCHOOL OF ENGINEERING

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

COCHIN – 682022



COMPUTATION METHODS

LABORATORY RECORD

Certified that this is the Bonafide Record of the work done by Sri/Smt. Jithika P. Sajan Admission No. 20522040 of Mechanical Engineering class in the Computational Methods Laboratory during the year 2024

Staff in- Charge Internal Examiner External Examiner

INDEX

SI.NO.	DATE	NAME OF EXPERIMENTS	PAGE NO.	REMARKS
1.	06/09/23	Review of Fundamentals of C ++	1	
2.	06/09/24	Matrix input & output		
3.	06/09/24	Vector operations		
4.	06/09/24	Matrix operations		
5.	06/09/24	Matrix & vector multiplication		
6.	27/09/24	Bisection method		
7.	27/09/24	Regula falsi		
8.				
9.				
10.				
11.				
12.				
13.				
14.				
15.				
16.				

Review of Fundamentals of c ++

1. C++ is a high-level, general-purpose programming language designed for system and application programming. It was developed by Bjarne Stroustrup at Bell Labs in 1983 as an extension of the C programming language. C++ is an object-oriented, multi-paradigm language that supports procedural, functional, and generic programming styles.
2. One of the key features of C++ is its ability to support low-level, system-level programming, making it suitable for developing operating systems, device drivers, and other system software. At the same time, C++ also provides a rich set of libraries and features for high-level application programming, making it a popular choice for developing desktop applications, video games, and other complex applications.
3. C++ has a large, active community of developers and users, and a wealth of resources and tools available for learning and using the language. Some of the key features of C++ include:
4. Object-Oriented Programming: C++ supports object-oriented programming, allowing developers to create classes and objects and to define methods and properties for these objects.
5. Templates: C++ templates allow developers to write generic code that can work with any data type, making it easier to write reusable and flexible code.
6. Standard Template Library (STL): The STL provides a wide range of containers and algorithms for working with data, making it easier to write efficient and effective code.
7. Exception Handling: C++ provides robust exception handling capabilities, making it easier to write code that can handle errors and unexpected situations.

Matrix input & output

Aim: To create a c++ program which inputs and then display a matrix of order $i \times j$

Algorithm.

1. Define the variables used.
2. Ask for inputs and read it
3. Declare the conditions using while loop
4. Show the output

Program

```
#include <iostream>
using namespace std;
int main() {
    int i = 0, j = 0;
    cout << "Enter number of rows: ";
    cin >> i;
    cout << "Enter number of columns: ";
    cin >> j;
    float l[i][j];
    int p = 0, q = 0;
    while (p < i) {
        while (q < j) {
            cout << "Enter element (" << p + 1 << ", " << q + 1 << "): ";
            cin >> l[p][q];
            q = q + 1;
        }
        p = p + 1;
        q = 0;
    }
    cout << "The entered matrix is:" << endl;
    for (p = 0; p < i; p++) {
        for (q = 0; q < j; q++) {
            cout << l[p][q] << "\t";
        }
        cout << endl;
    }
```



```
}  
    return 0;  
}
```

Result

Enter number of rows: 2

Enter number of columns: 2

Enter element (1, 1): 1

Enter element (1, 2): 2

Enter element (2, 1): 3

Enter element (2, 2): 4

The entered matrix is:

1 2

3 4

Vector Operations

Aims: To create a c++ programs for vector addition and multiplication.

Algorithm

Define the variables

Enter the inputs and read it

Enter Define the vector operations.

show the result.

Program

```
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cout << "Enter the first number: ";
    cin >> a;
    cout << "Enter the second number: ";
    cin >> b;
    int c = a + b;
    int d = a * b;
    cout << "The sum is: " << c << endl;
    cout << "The product is: " << d << endl;
    return 0;
}
```

Result

Enter the first number: 1

Enter the second number: 2

The sum is: 3

The product is: 2

Matrix Operation.

Aim: To create a C++ program to find the sum, product y transpose of matrices.

Algorithm.

- 1.Start
2. Input the dimensions (rows y columns) y elements for matrix A
- 3.Input the dimensions (rows & columns) y elements for matrix B
- 4.Check if matrix addition is possible. If possible, performs addition
5. check if matrix multiplication is possible. If possible, perform matrix multiplication.
6. Transpose matrix A & B by swapping its rows & columns.
7. Output results for matrix addition, multiplication & transpose.
- 8.Stop.

```

#include <iostream>

using namespace std;

void addMatrices(int A[10][10], int B[10][10], int rows, int cols) {
    int sum[10][10];
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            sum[i][j] = A[i][j] + B[i][j];
        }
    }

    cout << "Matrix addition:" << endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << sum[i][j] << "\t";
        }
        cout << endl;
    }
}

void transposeMatrix(int A[10][10], int rows, int cols) {
    int transpose[10][10];
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            transpose[j][i] = A[i][j];
        }
    }

    cout << "Transpose of the matrix:" << endl;
    for (int i = 0; i < cols; i++) {

```

```

        for (int j = 0; j < rows; j++) {
            cout << transpose[i][j] << "\t";
        }
        cout << endl;
    }
}

```

```

int main()
{
    int rows, cols;
    int A[10][10], B[10][10];
    cout << "Enter the number of rows and columns for matrix A: ";
    cin >> rows >> cols;
    cout << "Enter the elements of matrix A:" << endl;
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cin >> A[i][j];
        }
    }
    cout << "Enter the elements of matrix B:" << endl;
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {

```

```
        cin >> B[i][j];
    }
}
addMatrices(A, B, rows, cols);
transposeMatrix(A, rows, cols);
return 0;
}
```

Result

Enter the number of rows and columns for matrix A: 2 2

Enter the elements of matrix A:

1 2

3 4

Enter the elements of matrix B:

2 2

4 1

Matrix addition:

3 4

7 5

Transpose of the matrix:

1 3

2 4

Matrix and vector multiplication

Aims: To create a c++ program to find the product of a matrix and vector

Algorithm.

- 1.start
- .2.Input the number of rows and columns for matrix.
- 3.Declare a matrix, a vector and a result array initialized to zero.
- 4.Input the elements of matrix. using for loop.
- 5.Input elements of the vector using for loop.
- 6.Perform matrix multiplication.
- 7.Output the result
- 8.End.

Program

```
#include <iostream>
using namespace std;
int main()
{
    int rows, cols;
    cout << "Enter the number of rows and columns for the matrix: ";
    cin >> rows >> cols;
    int matrix[10][10];
    int vector[10];
    int result[10] = {0};
    cout << "Enter elements of the matrix:" << endl;
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cin >> matrix[i][j];
        }
    }
    cout << "Enter elements of the vector:" << endl;
    for (int i = 0; i < cols; i++) {
        cin >> vector[i];
    }
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result[i] += matrix[i][j] * vector[j];
        }
    }
}
```

```
    }  
}  
cout << "Resulting vector after multiplication: ";  
for (int i = 0; i < rows; i++) {  
    cout << result[i] << " ";  
}  
cout << endl;  
return 0;  
}
```

Result

Enter the number of rows and columns for the matrix: 2 2

Enter elements of the matrix:

1 2

3 4

Enter elements of the vector:

1 2

Resulting vector after multiplication: 5 11

Bisection method.

Aim: To write a program for a given function using bisection method.

Algorithm

1. Start
2. Define Function $F(x)$ $F(x)$ $F(x)$:
 - Input: A number xxx
 - Output: Calculate $F(x)=x^2-4x-10$ $F(x) = x^2 - 4x - 10$ $F(x)=x^2-4x-10$ and return the result.
3. Input Parameters:
4. Check if Roots are Bracketed:
5. Initialize Variables:
6. Bisection Loop:
7. Output the Results:
8. End

Program

```
#include <iostream>

#include <cmath>

using namespace std;

double F(double x) {
    return x * x - 4 * x - 10.0;
}

void bisection(double a, double b, int noit, double acc) {
    if (F(a) * F(b) > 0) {
        cout << "Roots are not bracketed by a and b." << endl;
        exit(0);
    }

    double xm;
    int n = 0;

    do {
        xm = (a + b) / 2.0;

        if (fabs(F(xm)) <= acc) {
            break;
        }

        if (F(xm) * F(a) < 0) {
            b = xm;
        } else {
            a = xm;
        }

        n += 1;
    } while (n < noit);

    cout << "Root of the equation is: " << xm << endl;
    cout << "Number of iterations is: " << n << endl;
}
```

```
int main() {  
    double a = 4.0;  
    double b = 6.0;  
    double acc = 0.00001;  
    int noit = 30;  
    bisect(a, b, noit, acc);  
    return 0;  
}
```

Result

Root of the equation is: 5.74166

Number of iterations is: 18

Regula Falsi

Aim: To create a c++ program for a given function using regula falsi method

Algorithm

1.Start

2.Define Function $F(x)$):

- Input: A number xxx
- Output: Compute $F(x)=x^2-4x-10$ and return the result.

3.Input Parameters:

4.Check if Roots are Bracketed:

5.Initialize Variables:

6.Regula Falsi Loop:

7.Convergence Check:

8. End

Program

```
#include <iostream>

#include <cmath>

using namespace std;

double F(double x)
{
    return (x * x) - (4 * x) - 10;
}

void regulaFalsi(double a, double b, int noit, double acc)
{
    if (F(a) * F(b) >= 0)
    {
        cout << "Roots are not bracketed by a and b" << endl;
        return;
    }

    double xm = a;

    int n = 0;

    for (n = 1; n <= noit; n++)
    {
        xm = a - (F(a) * (b - a)) / (F(b) - F(a));

        if (fabs(F(xm)) <= acc)
        {
            cout << "Root found at x = " << xm << " after " << n << " iterations" <<
endl;

            return;
        }
    }
}
```



```

        if (F(xm) * F(a) < 0)
        {
            b = xm;
        }
        else
        {
            a = xm;
        }
    }

    cout << "The method failed to converge after " << noit << " iterations." <<
endl;

    cout << "Approximate root after max iterations: " << xm << endl;
}

int main()
{
    double a = 4.0, b = 6.0;
    double acc = 0.00001;
    int noit = 30;
    regulaFalsi(a, b, noit, acc);
    return 0;
}

```

Newton Raphson

Aim: To create a c++ program for a given function using newton raphson method

Algorithm

Input: Initial guess x_0 , accuracy acc, max iterations noit.

Steps:

1. Check Initial Guess:
2. Iterate (up to noit):
3. Termination:

PROGRAM

```
#include <iostream>

#include <cmath>


$$F(x) = x^2 - 4x - 10$$


double F(double x)
{
    return x * x - 4 * x - 10;
}


$$dF(x) = 2x - 4$$


double dF(double x)
{
    return 2 * x - 4;
}

void NewtonRaphson(double x0, int noit, double acc)
{
    if (std::abs(F(x0)) <= acc)
    {
        std::cout << "Root found at x = " << x0 << " (Initial guess is close enough)"
        << std::endl;
        return;
    }

    double xm;

    int n = 0;

    while (n < noit)
    {
        xm = x0 - F(x0) / dF(x0);
```

```

        if (std::abs(F(xm)) <= acc)
        {
            std::cout << "Root found at x = " << xm << " after " << n + 1 << "
iterations" << std::endl;

            return;
        }

        x0 = xm;

        ++n;
    }

    std::cout << "Root not found within the given accuracy after " << noit << "
iterations." << std::endl;

    std::cout << "Closest approximation: x = " << xm << ", F(x) = " << F(xm) <<
std::endl;
}

int main()
{
    double x0 = 5.0;
    double acc = 0.00001;
    int noit = 30
    NewtonRaphson(x0, noit, acc);
    return 0;
}

```

Trapezoidal method

Aim: To create a c++ program for the given function using trapezoidal method.

Algorithm

1. Start
2. Define the function
3. Declare the variables n, a, b, s
4. Read the input values for a, b, n from user
5. Call the function `trapzd(a,b,n)` store results in s
6. Print the integral result "integral =" followed by value of s
7. Initialize variables of trapezoidal function
8. If $n=1$, return s
9. Else for $j=1$ to $n-1$, calculate $x=a+j*\text{del}$
10. Accumulate sum with $F(x)$
11. Return s
12. Calculate y & return it
13. End

Program

```
#include <iostream>

using namespace std;

float F(float x) {
    float y;
    y = 1.0 / (1.0 + x + x * x + x * x * x);
    return y;
}

float trapzd(float a, float b, int n) {
    float x, s, sum, del;
    int j;
    del = (b - a) / n;
    s = 0.5 * del * (F(a) + F(b));
    sum = 0;
    if (n == 1) {
        return s;
    } else {
        for (j = 1; j < n; j++) {
            x = a + j * del;
            sum += F(x);
        }
        s += del * sum;
        return s;
    }
}

int main()
```

```
{  
    float a, b;  
    int n;  
    cout << "Enter the limits a and b: ";  
    cin >> a >> b;  
    cout << "Enter the number of intervals n: ";  
    cin >> n;  
    float result = trapzd(a, b, n);  
    cout << "The integral = " << result << endl;  
    return 0;  
}
```

Result

Enter the limits a and b: 0 1

Enter the number of intervals n: 4

The integral = 0.569247

Simpsons 3/8th rule

Aim : To create a c++ program for the given function using Simpsons 3/8th rule.

Algorithm

1. Input:

- Accept the lower limit a, upper limit b, and number of subintervals n (note: n must be a multiple of 3).

2. Check Subinterval Condition:

- If $n \bmod 3 \neq 0$, display an error message: "Number of subintervals must be a multiple of 3" and stop.

3. Calculate Step Size:

4. Initialize Sum:

5. Iterate Over Subintervals:

6. Calculate Result:

Program

```
#include <iostream>
using namespace std;
float f(float x)
{
    return 1.0 / (1.0 + x + x * x + x * x * x);
}
float simpsons_3_8(float a, float b, int n)
{
    if (n % 3 != 0)
    {
        cout << "Error: Number of subintervals must be a multiple of 3." << endl;
        return 0.0;
    }
    float h = (b - a) / n;
    float sum = f(a) + f(b);
    for (int i = 1; i < n; i++)
    {
        float x = a + i * h;
        if (i % 3 == 0)
        {
            sum += 2 * f(x);
        }
        else
        {
            sum += 3 * f(x);
        }
    }
    float result = (3 * h / 8) * sum;
    return result;
}
int main()
{
    float a, b;
```

```

int n;
cout << "Enter the lower limit of interval (a): ";
cin >> a;
cout << "Enter the upper limit of interval (b): ";
cin >> b;
cout << "Enter the number of subintervals (must be a multiple of 3): ";
cin >> n;
if (n % 3 == 0)
{
    float result = simpsons_3_8(a, b, n);
    cout << "The result of integration using Simpson's 3/8 rule is: " << result <<
endl;
}
else
{
    cout << "Error: Number of subintervals must be a multiple of 3." << endl;
}
return 0;
}

```

Result

```

Enter the lower limit of interval (a): 1
Enter the upper limit of interval (b): 4
Enter the number of subintervals (must be a multiple of 3): 12
The result of integration using Simpson's 3/8 rule is: 0.193387

```

Simpsons 1/3rd rule

Aim: To create a c++ program for the given function using Simpsons 1/3rd rule.

Algorithm

1. Input
2. Check Interval Condition
3. Calculate Interval Width
4. Initialize Sums
5. Sum Terms with Coefficient 4
6. Sum Terms with Coefficient 2
7. Calculate Result Using Simpson's 1/3 Rule

Program

```
#include <iostream>
using namespace std;
float F(float x)
{
    return x + x * x;
}
float simpson_1_3(float a, float b, int n)
{
    if (n % 2 != 0)
    {
        cout << "Error: Number of intervals must be even." << endl;
        return 0.0;
    }

    float del = (b - a) / n;
    float sum1 = 0.0;
    float sum2 = 0.0;
    float x = a + del;
    for (int i = 1; i < n; i += 2)
    {
        sum1 += F(x);
        x += 2 * del;
    }
    x = a + 2 * del;
    for (int i = 2; i < n; i += 2)
    {
        sum2 += F(x);
        x += 2 * del;
    }
}
```

```

    }
    float s = (del / 3) * (F(a) + F(b) + 4 * sum1 + 2 * sum2);
    return s;
}
int main()
{
    float a, b;
    int n;
    cout << "Enter the lower limit of the interval (a): ";
    cin >> a;
    cout << "Enter the upper limit of the interval (b): ";
    cin >> b;
    cout << "Enter the number of intervals (n): ";
    cin >> n;
    if (n % 2 == 0)
    {
        float result = simpson_1_3(a, b, n);
        cout << "The value of the integral is: " << result << endl;
    }
    else
    {
        cout << "Error: Number of intervals must be even." << endl;
    }
    return 0;
}

```

Result

```

Enter the lower limit of the interval (a): 1
Enter the upper limit of the interval (b): 4
Enter the number of intervals (n): 4
The value of the integral is: 28.5

```

Gaussian Quadrature Formula

Aim: To create a c++ program for the given function using Gaussian Quadrature Formula

Algorithm

1. Define the limits of integration, a and b.
2. Define the function $f(x)$ to be integrated.
1. Initialize Gauss Points and Weights:
 - Use a predefined set of Gauss points $x[]$ and weights $w[]$ for the 3-point Gauss quadrature:
 - Gauss points: $x=\{0.0, -0.7745966692, 0.7745966692\}$
 - Weights: $w=\{8/9, 5/9, 5/9\}$
2. Set Initial Sum:
 - Initialize the sum $s=0.0$ to store the accumulated value of the weighted function evaluations.
3. Loop Over Gauss Points:
4. Compute the Final Integral:

Program

```
#include <iostream>
#include <cmath>
using namespace std;
#define F(x) ((x) * (x) * (x) * (x))
float gausq(float a, float b)
{
    int j;
    float s = 0.0;
    static float x[] = {0.0, -0.7745966692, 0.7745966692};
    static float w[] = {8.0 / 9.0, 5.0 / 9.0, 5.0 / 9.0};
    for (j = 0; j < 3; j++)
    {
        s += w[j] * F(0.5 * (b - a) * x[j] + 0.5 * (b + a));
    }
    return 0.5 * (b - a) * s;
}

int main()
{
    float a, b, intg;
    cout << "Enter the limits a and b: ";
    cin >> a >> b;
    intg = gausq(a, b);
    cout << "Integral = " << intg << endl;
    return 0;
}
```

Result

Enter the limits a and b: 0 1

Integral = 0.2